WEEK 2 · 16–19 NOVEMBER 2026

Hyperspectral image processing workflows

From a pile of cells to a pipeline you can rerun, batch and defend

Week 2 · Session 1 · practical: 12_processing_workflow.ipynb

LEARNING OBJECTIVES

What you will be able to do

- Describe the standard processing chain and the role of each step
- Drive an analysis from a **configuration**, so no parameter is buried in code
- Export analysis-ready products: a Cloud-Optimised GeoTIFF plus a provenance sidecar
- Batch-process many scenes and read the summary table like a quality analyst
- Process a full scene **tile by tile** with bounded memory, and know when to reach for dask
- Apply the reproducibility checklist your project submission will be held to

1. Why a workflow

The difference between an analysis and a result

THE STANDARD TO HOLD YOURSELF TO

A result you cannot reproduce is not a result.

It is an anecdote with a colour bar. The same is true of a map whose parameters exist only in the scroll history of a notebook nobody else can run.

Symptoms of a pile of cells

- The same twelve steps copy-pasted into six notebooks, each slightly different
- A threshold changed in one place but not another
- A mask applied in one notebook and forgotten in the next
- Cells run out of order, so the file no longer describes what actually happened
- Nobody can say which version of the code produced the figure in the report

What makes it a workflow

Property	In practice
Parameterised	everything that can change lives in one configuration object, not scattered in code
Modular	each step is a function with clear inputs and outputs, so it can be tested and replaced
Traceable	every output records how it was made: scene, parameters, software versions, date

The course processing chain

- **1. Find / load** — STAC or local HDF5 · `hu.get_scene`
- **2. QA masks** — no-data, cloud, cirrus · `scene.masks`, `scene.valid_mask`
- **3. Bad bands** — water vapour, SWIR edge, low SNR · `hu.good_band_mask`
- **4. Subset** — window or tiles · `read_cube(window=…)`
- **5. Analyse** — indices, SAM, classification · `hu.nd`, `hu.spectral_angle`
- **6. Export** — COG plus JSON sidecar · `hu.save_geotiff`

Library code and notebook code

- **`hsi_utils.py`** holds the reusable chain: loading, masking, bands, export, spectral helpers
- The **notebook** tells the story: why these parameters, what the figures show
- Later, `app/mangrove_product.py` takes the final method out of the notebook entirely
- A notebook cannot be imported, scheduled, unit-tested or reviewed line by line
- **Rule of thumb:** if you have pasted it twice, it belongs in a module

2. Anatomy of a pipeline

Configuration, steps, gates, outputs

One dictionary holds every choice

```
CONFIG = {
    "scene_id": "20250608_091605_90_4001",   # Konya, Türkiye
    "window": (300, 500, 400, 600),          # None = whole scene
    "mask_cloud": True,
    "min_valid_fraction": 0.5,               # QA gate
    "indices": {                             # name: (band_a_nm, band_b_nm)
        "NDVI": (860, 665), "NDRE": (860, 705),
        "NDWI": (560, 860), "NDMI": (860, 1650), "NBR": (860, 2200),
    },
    "output_dir": hu.OUTPUT_DIR / "12",
    "overwrite": True,
}
```

To process another area you change the configuration, not the code. In a project, keep it in YAML or JSON under version control.

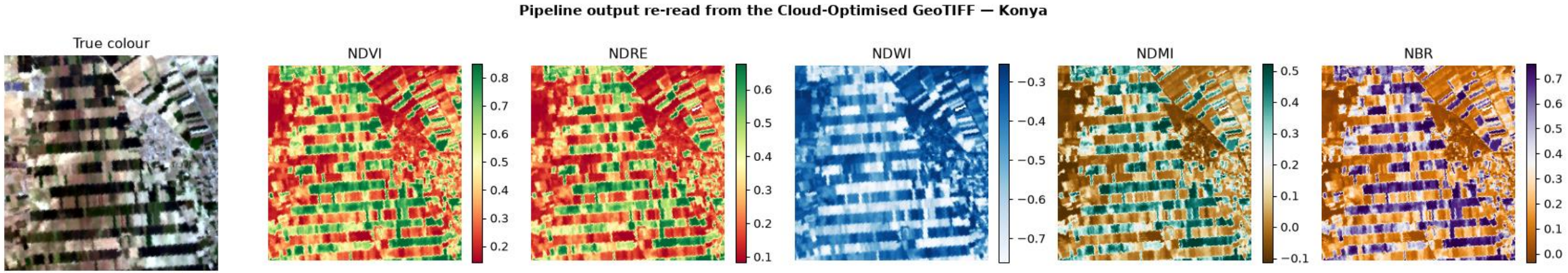
Steps as functions, with a gate

- Each step takes the config plus the previous result, and returns something the next step can use
- The **QA step is a gate**: if too few pixels are usable, the pipeline stops with a clear message
- A window in the no-data corner gave `valid_fraction = 0.011` and was rejected at the 50 % gate
- Failing loudly beats quietly producing a map of NaNs
- Gates belong wherever an assumption could silently break downstream

Logging instead of print

- A pipeline that runs unattended needs a **time-stamped log**, on screen and in a file
- Log the scene, the QA fractions, what was computed, what was written, and how long it took
- Log **skips** too: a scene not on disk, an output that already exists
- `logging` gives you levels — INFO for progress, WARNING for a QA gate that stopped a run
- The log is the first thing you read when a batch of 50 scenes finishes overnight

Re-open what you wrote, and look at it



The exported COG read back: 5 bands × 200 × 200, EPSG:32636, band names NDVI · NDRE · NDWI · NDMI · NBR

Analysis-ready output = data + provenance

- A **Cloud-Optimised GeoTIFF**: internally tiled with overviews, streamable by QGIS and web maps
- **Band names** written into the file, so nobody has to guess what band 3 is
- A **JSON sidecar** with scene ID, acquisition time, CRS, geotransform and the exact band centres used
- Plus QA fractions, software versions, a hash of `hsi_utils.py`, and the data attribution
- Months later, that sidecar is the difference between a reusable product and a mystery raster

The sidecar that travels with the product

```
{
  "scene_id": "20250608_091605_90_4001",
  "acquired_utc": "2025-06-08T09:16:05+00:00",
  "crs": "EPSG:32636",
  "band_centres_used_nm": {"665": 665.87, "860": 861.26, "1650": 1647.66},
  "qa": {"valid_fraction": 0.999, "cloud_flag_fraction": 0.001},
  "software": {"python": "3.13.13", "numpy": "2.5.3",
               "hsi_utils_sha256": "5efdf1ca2448"},
  "attribution": "Tanager STAC Data ... (c) 2025 Planet Labs PBC. CC-BY-4.0"
}
```

Idempotent re-runs

- Running the same step twice with the same input should give the same result
- …and should not redo expensive work unless you ask for it
- With `overwrite=False` the export step sees the file and skips it: 0.4 s becomes 0.1 s
- This is what makes a 50-scene batch restartable after a crash
- It also stops you silently overwriting the figure that is already in your report

3. Scaling up

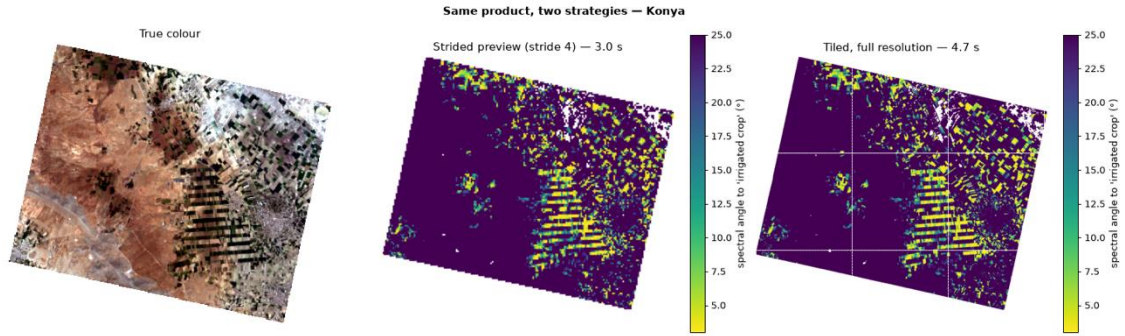
Memory, tiles and where the time actually goes

Do the memory arithmetic first

- Konya scene: 426 bands × 673 × 776 px × 4 bytes = **0.89 GB** as float32
- The larger Malibu scene: 426 × 1063 × 957 = **1.73 GB**
- NumPy often makes one or two temporary copies during a calculation
- A 256 × 256 tile of all bands is only **112 MB**
- Free Colab gives you ~12 GB of RAM — a stack of scenes will not fit

Tiling: full resolution, bounded memory

- A generator yields one window at a time: 12 tiles of up to 256 × 256 px
- Full-resolution spectral angle in **4.7 s**, peak memory ≈ **368 MB**
- Two empty tiles skipped by a cheap validity check first
- Notebook 30 does the same over a 1,063 × 957 scene: 8.3 s, **0.24 GB** peak
- Small angles (yellow) pick out every irrigated field



True colour · strided preview · tiled full resolution, with tile boundaries

Two surprises worth knowing

- **Striding does not save time.** Tanager HDF5 is stored in compressed chunks of 14 bands × 44 × 49 px
- Reading every 4th pixel still decompresses every chunk: 3.0 s strided vs 4.7 s at full resolution
- Striding saves **memory**, not disk I/O — for fast previews use the `ortho_visual` COG or a subset
- **Neighbourhood operations need a halo.** Filters, texture, MNF noise, CNN patches
- Without overlapping tiles you get visible seams at every tile boundary

OPTIONAL, BUT USEFUL

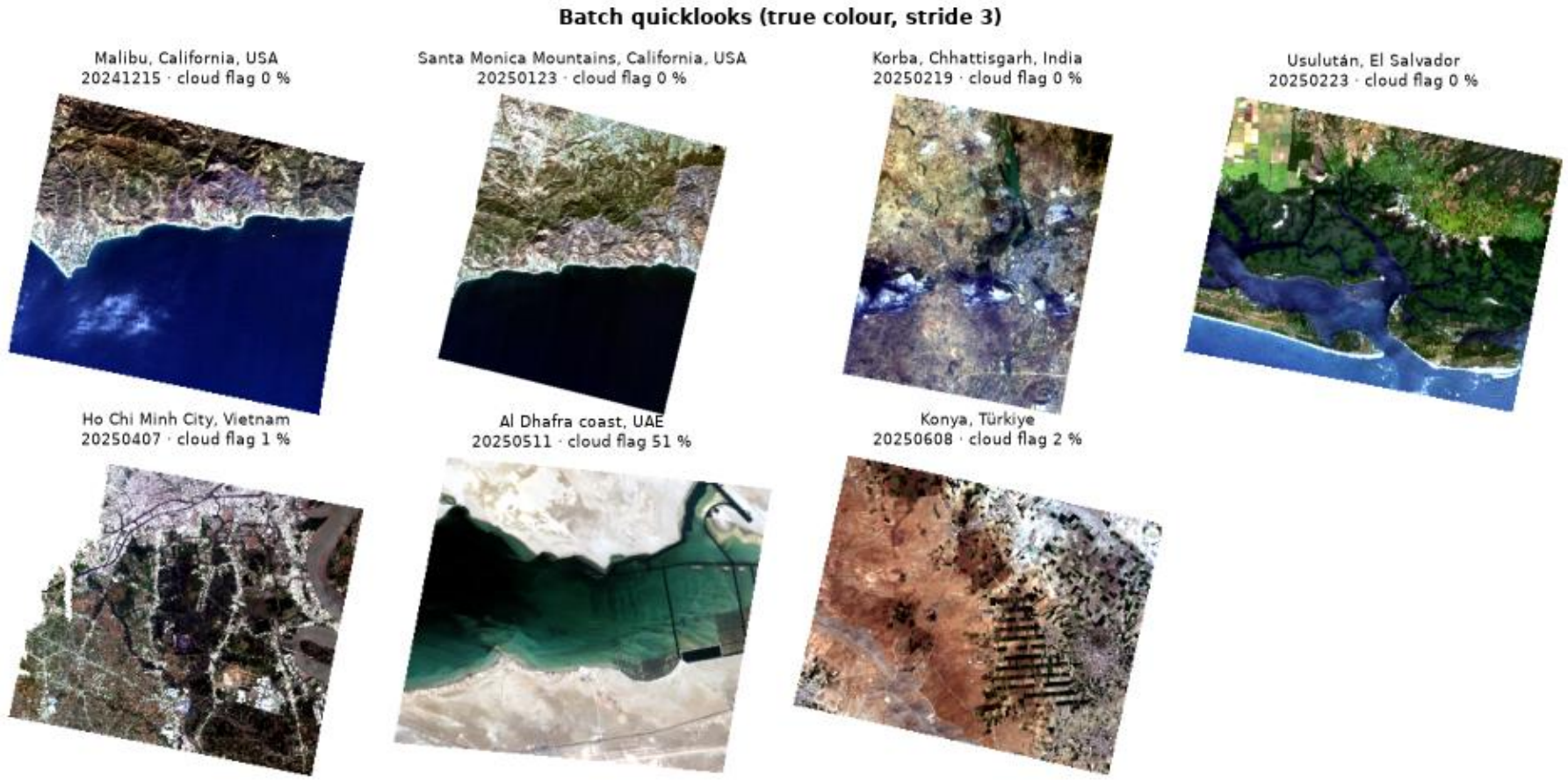
When to reach for xarray and dask

- **xarray** attaches coordinates and a CRS: select by *wavelength*, not band index
- `da.sel(wavelength=665, method="nearest")` instead of remembering band 58
- **dask** builds a lazy graph and processes chunk by chunk, in parallel if you have cores
- Full-scene mean reflectance computed lazily in **4.4 s**, identical to the tiled result
- Use them when the data no longer fits, or when you want the tiling done for you

4. Batch and reproducibility

Many scenes, and the habits that make them trustworthy

Seven scenes, one loop, one table



Batch quicklooks with the cloud-flag percentage under each scene

Read the batch table like a QA analyst

- One row jumps out: **Al Dhafra** has a very high cloud-flag percentage
- Its quicklook is almost cloud-free — the beta mask confuses bright sabkha with cloud
- A pipeline that trusted the mask would silently delete half the land in that scene
- Strided reads are fine for **statistics**; you do not need full resolution for a summary
- **Always** pair the table with quicklooks before trusting a batch result

The reproducibility checklist

Practice	How
Pin versions	requirements.txt, and store the versions in the sidecar
Seed randomness	np.random.default_rng(SEED), random_state=SEED, torch.manual_seed
Config, not constants	one config dict or file per run, saved next to the output
Provenance	JSON sidecar: scene, parameters, versions, date, attribution
Log	one log file per run; warnings for skips and QA failures
Idempotent	rerunning does not change or duplicate outputs
Restart & Run All	before every submission, on a clean kernel
Relative paths	hu.DATA_DIR and hu.OUTPUT_DIR, never /Users/yourname/…

Determinism is testable

- Run the same computation twice with the same seed and compare a hash of the result
- Same seed → identical digest; different seed → a different sample, as expected
- If a 'deterministic' step gives two answers, you have an unseeded random call or thread non-determinism
- Cheap to write, and it catches a class of bug that is otherwise invisible

PRACTICAL

Over to you

▶ `12_processing_workflow.ipynb` · `120 min`

- Run the pipeline on a window, inspect the COG and the sidecar it wrote
- Trigger the QA gate deliberately with a no-data window and read the message
- Batch all local scenes and find the anomaly in the summary table
- Process a full scene tile by tile and compare time and memory with a strided read
- Exercise 3: add a halo so a neighbourhood operation has no seams

KEY TAKEAWAYS

Recap

- A workflow is **parameterised, modular and traceable** — config, functions, provenance
- Export **analysis-ready** products: COG with band names plus a JSON sidecar
- **Gate** your assumptions and **log** what happened; make reruns idempotent
- Tiling keeps memory at one tile: 0.89 GB cube processed with 368 MB peak
- Striding saves memory, not I/O — compressed chunks are read whole
- Neighbourhood operations need a **halo**; dask automates the chunking when you outgrow a laptop
- Reproducibility is a checklist, not a talent: seeds, versions, logs, relative paths

Next: Session 2 — Feature extraction and dimensionality reduction: what to compute inside step 5

References and further reading

Wilson, G. et al. (2017). Good enough practices in scientific computing. PLOS Computational Biology 13, e1005510.

Sandve, G.K. et al. (2013). Ten simple rules for reproducible computational research. PLOS Computational Biology 9, e1003285.

Rocklin, M. (2015). Dask: parallel computation with blocked algorithms and task scheduling. SciPy 2015.

Hoyer, S. & Hamman, J. (2017). xarray: N-D labeled arrays and datasets in Python. JORS 5, 10.

Cloud-Optimised GeoTIFF specification — cogeo.org

Course material: notebook 12_processing_workflow.ipynb · hsi_utils.py · templates/interim_qa_checklist.md